

An Ontology of Hardware Electronic Music Instruments

Roberto Chiurazzi

Queen Mary University of London

`r.chiurazzi@se24.qmul.ac.uk`

May 2026

Abstract

This report presents an OWL2 ontology covering hardware electronic music instruments: synthesizers, samplers, and drum machines. Two parallel data-integration pipelines were implemented to populate the ontology. The first reads from a manually verified CSV produced by scraping the Vintage Synth Explorer catalogue and cross-checking each entry against DBpedia. The second queries the DBpedia SPARQL endpoint at runtime for every candidate. The T-Box organises instruments by manufacturer, country, year of release, sound generation paradigm, and synthesis method, with several defined classes and inferred properties derived from these basic facts. Reasoning is performed with Pellet, and a set of SWRL rules extends the ontology with temporal classification by decade. The two pipelines share a single T-Box but produce A-Boxes of different size and quality, with 142 verified instances against 371 unverified ones, illustrating a recurring trade-off in semantic data integration between manual curation and automated population. A companion Python script demonstrates the system through seven queries that exercise the ontology's inferred structure.

1 Introduction

This report describes the Electronic Music Instrument Ontology¹, an OWL2 ontology covering hardware synthesizers, samplers, and drum machines from the 1970s onwards. The domain was chosen for two related reasons. Musical instruments in general have been studied as a knowledge representation problem since at least Hornbostel and Sachs' 1914 classification, and that line of work has been revisited many times in the Semantic Web era [1, 2]; tackling the full domain in a coursework of this scope would have been impractical. Hardware electronic music instruments are, by contrast, a younger and less formalised corner of the same domain, and one in which the author already has practical and theoretical familiarity. This familiarity shaped several modelling decisions, most notably the choice to keep sound generation (analog, digital, hybrid) and synthesis method (subtractive, FM, sample-based, and so on)

as orthogonal properties, instead of collapsing them into a single classification.

Reasoning is performed with Pellet, chosen because the ontology uses SWRL numeric built-ins for decade classification.

Domain background

A hardware synthesizer is a self-contained electronic instrument that generates and shapes sound through one of several synthesis paradigms. Two of the most common paradigms are: *Subtractive synthesis*, in which a harmonically rich oscillator signal is shaped by a filter, and *FM (frequency modulation) synthesis*, in which the frequency of one oscillator modulates another to produce complex timbres. Other methods present in the ontology are sample-based, wavetable, vector, additive, and physical modelling synthesis. Orthogonal to the synthesis method is the signal path: *analog* synthesizers process voltages through electronic circuits, *digital* synthesizers compute sound samples in software or DSP hardware, and *hybrid* designs combine the two. Two related instrument families share much of this vocabulary: *samplers*, which record short audio fragments and trigger playback under keyboard or MIDI control, and which may use digital or hybrid sound generation paradigms; and *drum machines*, which generate or sequence percussive sounds and can be analog (e.g. the Roland TR-808), digital (e.g. the Linn Drum), or hybrid (e.g. the Roland TR-909).

The rest of this report is organised as follows. Section 2 describes the ontology design, covering classes, properties, defined classes, and SWRL rules. Section 3 details the data sources used and the two population pipelines. Section 4 covers reasoning and the seven competency queries written for the system. Section 5 compares the two pipelines quantitatively. Section 6 reflects on the engineering decisions and limitations encountered during development. Section 7 situates the work within the broader literature on ontology engineering and modern AI approaches to data semantics.

2 Ontology design

2.1 Classes

The class hierarchy is shown in Table 1. The top concept is `HardwareInstrument`, with three disjoint

¹http://qmul.ac.uk/ecs7028/electronic_music_instrument_ontology

subclasses: **Synthesizer**, **Sampler**, and **DrumMachine**. Four supporting classes describe entities that instruments are related to: **Manufacturer**, **Country**, **SoundGeneration**, and **SynthesisMethod**. A **Decade** class is included to hold a fixed set of named decade individuals.

Class	Parent	Status
HardwareInstrument	—	asserted
Synthesizer	HardwareInstrument	asserted
Sampler	HardwareInstrument	asserted
DrumMachine	HardwareInstrument	asserted
Manufacturer	—	asserted
Country	—	asserted
SoundGeneration	—	asserted
SynthesisMethod	—	asserted
Decade	—	asserted
AnalogInstrument	HardwareInstrument	inferred
DigitalInstrument	HardwareInstrument	inferred
HybridInstrument	HardwareInstrument	inferred
AnalogSynth	Synthesizer	inferred
DigitalSynth	Synthesizer	inferred
HybridSynth	Synthesizer	inferred
PolyphonicSynth	Synthesizer	inferred
MonophonicSynth	Synthesizer	inferred
AnalogDrumMachine	DrumMachine	inferred
DigitalDrumMachine	DrumMachine	inferred
HybridDrumMachine	DrumMachine	inferred

Table 1: Asserted and inferred classes.

Three supporting classes have a fixed set of named individuals asserted in the T-Box. **SoundGeneration** has the three mutually exclusive individuals **Analog**, **Digital**, and **Hybrid**. **SynthesisMethod** has seven individuals covering the methods listed in the introduction. **Decade** has five individuals, **Seventies** through **TwoThousandTens**, explicitly declared as pairwise different so the reasoner treats them as distinct. Modelling decades as individuals of a single class rather than as subclasses of **HardwareInstrument** reflects that a decade is a time period an instrument belongs to, not a kind of instrument.

2.2 Object and datatype properties

The ontology defines seven object properties and two datatype properties, summarised in Table 2. Several of the object properties are functional, expressing that each instrument has at most one value: one manufacturer, one sound generation paradigm, and one decade. The **contemporaryOf** property is symmetric and irreflexive, since "contemporary with" is bidirectional and an instrument cannot be contemporary with itself. The **originCountry** property is not asserted directly: it is computed by the reasoner from **hasManufacturer** and **basedIn**, so the country attributed to an instrument follows from the country of its manufacturer.

hasSynthesisMethod is left non-functional because some hybrid digital instruments combine more than one synthesis approach. **polyphony** has its domain re-

Property	Characteristics
hasManufacturer	Functional
basedIn	Functional
originCountry	Inferred via property chain over hasManufacturer and basedIn
hasSoundGeneration	Functional
hasSynthesisMethod	—
contemporaryOf	Symmetric, Irreflexive
hasDecade	Functional, populated by SWRL
yearOfRelease	Functional, integer
polyphony	Functional, integer; domain Synthesizer

Table 2: Object and datatype properties.

stricted to **Synthesizer** because voice count is mostly a defining feature of synthesizers.

2.3 Defined classes and DL

Eleven classes in the ontology are *defined*: their membership is computed by the reasoner from equivalent-class axioms rather than asserted directly. Their definitions are given in Table 3.

The three cross-cutting classes (**AnalogInstrument**, **DigitalInstrument**, **HybridInstrument**) classify any instrument by its sound generation paradigm, regardless of whether it is a synthesizer, sampler, or drum machine. The synth-specific and drum-machine-specific defined classes intersect that classification with the top-level instrument type, so that a synthesizer asserted as **Analog** ends up in both **AnalogInstrument** and **AnalogSynth** after reasoning. **Sampler** is left unconstrained at the class level; every sampler in the A-Box is digital, but no class axiom forces this, leaving room for hybrid samplers.

2.4 SWRL rules

Five SWRL rules in the T-Box classify each instrument into a decade based on its release year. They use the SWRL built-in atoms **swrlb:greaterThanOrEqual** and **swrlb:lessThan**, which are not expressible in pure OWL2 DL. The pattern is the same for all five; the Eighties rule reads:

$$HdwInst(?x) \wedge year(?x, ?y) \wedge (?y \geq 1980) \wedge (?y < 1990) \rightarrow hasDecade(?x, Eighties)$$

The four remaining rules use the bounds 1970/1980, 1990/2000, 2000/2010, and 2010/2020.

2.5 External concept linking

The ontology declares twelve **skos:exactMatch** links to Wikidata: five at the class level (**Synthesizer**, **Sampler**, **DrumMachine**, **Subtractive**, and **FM**) and seven at the individual level for countries (Japan, United States, Italy, Germany, Sweden, Australia,

Class	DL definition
AnalogInstrument	<code>HardwareInstrument $\sqcap$ $\exists$hasSoundGeneration.{Analog}</code>
DigitalInstrument	<code>HardwareInstrument $\sqcap$ $\exists$hasSoundGeneration.{Digital}</code>
HybridInstrument	<code>HardwareInstrument $\sqcap$ $\exists$hasSoundGeneration.{Hybrid}</code>
AnalogSynth	<code>Synthesizer $\sqcap$ AnalogInstrument</code>
DigitalSynth	<code>Synthesizer $\sqcap$ DigitalInstrument</code>
HybridSynth	<code>Synthesizer $\sqcap$ HybridInstrument</code>
PolyphonicSynth	<code>Synthesizer $\sqcap$ (≥ 2polyphony)</code>
MonophonicSynth	<code>Synthesizer $\sqcap$ ($= 1$polyphony)</code>
AnalogDrumMachine	<code>DrumMachine $\sqcap$ AnalogInstrument</code>
DigitalDrumMachine	<code>DrumMachine $\sqcap$ DigitalInstrument</code>
HybridDrumMachine	<code>DrumMachine $\sqcap$ HybridInstrument</code>

Table 3: Defined classes and their description-logic definitions.

Russia). At the instance level, each instrument carries two annotation properties (`dbpediaRef` and `wikipediaRef`) holding URIs of its DBpedia and Wikipedia pages. The choice to use `skos:exactMatch` for abstract concepts and string annotations for instance references rather than `owl:sameAs` is discussed in Section 6.

3 Population and data integration

3.1 Choosing a data source

After defining the T-Box, the search for population data started with two music-specific candidates. MIMO (Musical Instrument Museums Online)² advertises a SPARQL endpoint that would, in principle, be the natural choice for an instrument ontology; in practice every query attempted during development returned HTTP 500 errors, making it unusable as a run-time source. DBpedia therefore became the best practical compromise: its coverage of well-known vintage instruments is good, infobox fields are reasonably uniform, and the public SPARQL endpoint³ is responsive.

DBpedia alone, however, is not a list. There is no canonical DBpedia resource that enumerates "all hardware synthesizers", so a secondary source was needed to provide candidate names. Vintage Synth Explorer (VSE)⁴ is a long-running online reference covering several hundred hardware electronic music instruments organised by manufacturer, with infobox-like sidebars holding model name, year, sound generation and synthesis type. VSE became the seed source for candidate names, with DBpedia providing the structured data once a candidate had been matched.

Using VSE involved web scraping. The scraper was kept simple: it issued one request per manufacturer page, with a short pause between requests to avoid placing unreasonable load on the server, and it only extracted the small set of fields needed for the ontology

rather than copying full pages. The data was used solely for this academic project.

3.2 First pipeline: curated CSV

The first pipeline produces a manually verified CSV from which the A-Box is then built.

The process began with a small Python scraper that walked the VSE manufacturer index and extracted basic attributes for each instrument, producing a CSV of 892 candidate entries.

For each candidate, a label-matching SPARQL query was issued to DBpedia to check whether a matching resource existed and, if so, to retrieve its release year, sound generation paradigm, synthesis type, polyphony and Wikipedia URL. About 416 of the 892 candidates returned a DBpedia match.

A short filtering script then kept only the entries with reasonably complete data, dropping candidates that were missing more than one or two of the relevant attributes. For the small remaining gaps, information was filled in by hand against the corresponding Wikipedia articles, and a handful of DBpedia inconsistencies were corrected at the same time: the Yamaha DX7, for instance, is listed in DBpedia as using Additive synthesis when it is in fact the canonical commercial FM synthesizer; the Oberheim OB-X has a polyphony value of 46 in DBpedia, which is implausible (the real value is 8) and was corrected.

The resulting file is a CSV of 142 verified instruments. A separate script (`populate.py`) then reads this CSV and writes the A-Box: it creates an individual for each instrument and manufacturer, asserts the appropriate properties, and writes the populated ontology back to the OWL file.

3.3 Second pipeline: runtime SPARQL

The second pipeline was developed as an empirical alternative to the curated CSV one. Instead of preparing a verified intermediate file, it queries DBpedia directly for each candidate at runtime and inserts whatever it finds into the A-Box, subject to validation.

²<https://data.mimo-db.eu/sparql/>

³<https://dbpedia.org/sparql>

⁴<https://vintagesynth.com>

The script reads the unverified seed list from the previous step. Wikipedia sometimes redirects two distinct synthesizers to the same article (the Roland JP-8000 and JP-8080, for example), which means both entries point to the same DBpedia resource; duplicates are removed before querying. One SPARQL query is then issued per remaining candidate. Validation is applied at two levels:

- **In the SPARQL query:** `FILTER REGEX` clauses reject year values that are not four-digit numbers and polyphony values that are not pure integer strings, screening out malformed infobox content such as date ranges or URL-typed polyphony fields.
- **In Python, after retrieval:** a year must parse to an integer between 1950 and 2026; a polyphony value must parse to an integer between 1 and 256; and a synthesis type must map onto one of the seven controlled-vocabulary individuals in the T-Box, with unrecognised values discarded rather than imported.

4 Reasoning and querying

A companion Python script, `query.py`, loads the populated ontology, applies a reasoning step, and evaluates seven competency queries. The queries are summarised in Table 4; their full SPARQL text is given in the script. Together they exercise property chains, defined classes, SWRL-inferred properties, and a range of SPARQL 1.1 features.

#	Query
Q1	Each manufacturer’s synthesis methods, with counts.
Q2	Synthesizers contemporary with the Roland Juno-60.
Q3	Instrument counts per sound generation paradigm.
Q4	Synthesizers with no asserted synthesis method (data audit).
Q5	Synthesis methods by decade.
Q6	Instrument counts per decade.
Q7	Synthesis methods used by Digital instruments.

Table 4: The seven queries.

Most of the queries produce results consistent with the historical development of electronic instruments, with subtractive synthesis dominating the 1970s, sample-based synthesis becoming prominent in the 1990s, and Yamaha’s catalogue being predominantly FM-based. Q7 produces a more interesting result. Filtering for instruments with Digital sound generation, seven synthesizers come back as using Subtractive synthesis: the Roland JP-8000 (1996) and JP-8080 (1998), the Yamaha AN1x (1997), the Korg MS2000 (2000), the Alesis Micron (2002) and Ion (2003), and

the Roland SH-201 (2006). All seven belong to the virtual-analog category, in which DSP-based digital instruments emulate the classic analog subtractive signal chain. The fact that the query surfaces this cluster cleanly is a small validation of the design choice to keep `hasSoundGeneration` and `hasSynthesisMethod` as orthogonal properties: collapsing them into a single classification would have made this observation inexpressible.

5 Data integration method evaluation

The two population pipelines share the same T-Box, so a single set of competency queries (Section 4) runs against either output. The pipelines differ in how the A-Box is constructed and, as a result, in its size, completeness, and reliability. Table 5 summarises the two outputs.

	Curated CSV	Runtime SPARQL
Instruments	142	371
Manufacturers	23	73
Total triples	1,981	4,380
Manual review	yes	no
DBpedia errors corrected	yes	no
Typical runtime	seconds	30–45 min

Table 5: Comparison of the two population pipelines.

The runtime SPARQL pipeline produces roughly $2.6\times$ the number of instruments, but at a real cost in data completeness. Of the 337 synthesizers among the 371 instruments in the SPARQL output, 189 (56%) have no synthesis method recorded, 185 (55%) have no sound generation paradigm, and 219 (65%) have no polyphony value, simply because the corresponding DBpedia infoboxes do not carry those fields. This shows up most directly in Q4: it returns zero results against the curated CSV, but 189 against the runtime SPARQL output.

DBpedia also carries richer fields such as the abstract paragraphs of each Wikipedia article. These were briefly fetched as local `rdfs:comment` annotations but dropped because the queries became too slow. A future, slower run could add them back, together with other types of properties if richer data is required.

Neither output is strictly better. The curated CSV is the right choice when correctness matters, while the runtime SPARQL output is the right choice when breadth matters: it covers a much larger fraction of the vintage synthesizer landscape with no manual effort, at the cost of accepting whatever DBpedia provides, including its gaps.

6 Discussion

A few engineering decisions taken during development are worth reflecting on. The Protégé reasoner used throughout was Pellet rather than HermiT, because HermiT does not support the `swrlb` numeric built-in atoms used by the decade rules. The `contemporaryOf` property was initially defined by a SWRL rule, but its symmetric pairwise expansion slowed classification enough that the property is now populated on demand through SPARQL (Q2) instead, with its formal characteristics still declared in the T-Box.

External concept linking went through one substantial revision. A first version used `owl:sameAs` between each instrument and its DBpedia resource. In the runtime SPARQL pipeline, where several pairs of synthesizers resolve to the same DBpedia article because Wikipedia redirects them together, the reasoner inferred that the two local individuals were the same; this then clashed with functional properties such as `polyphony` when the two had different asserted values, raising an inconsistency. A stricter linked-data design would have kept `owl:sameAs` and added an extra step that handles such cases, either by joining the two synths into a single individual or by skipping one.

The present work therefore settled on `skos:exactMatch` for class-level and country-level external links, and annotation properties (`dbpediaRef`, `wikipediaRef`) holding the URIs as literal values for instance-level references. `skos:exactMatch` is a weaker commitment than `owl:sameAs` (referring to the same thing rather than being the same thing), but it prevents the inconsistencies seen earlier and lets the reasoner ignore the per-instance URIs as data. This is a pragmatic compromise rather than the strongest linked-data design, and revisiting the cleaner `owl:sameAs` approach with proper handling of shared articles would be one direction for future iterations.

The ontology in its current form is a small, focused case study. Its scope is limited to hardware instruments up to the mid-2010s, and the A-Box is small enough that the reasoner runs in seconds. The most natural next step would be to extend the same T-Box to software synthesizers, which share most of the modelled vocabulary while differing in distribution and platform. Connecting the instruments here to the wider context in which they are actually used is a harder problem: the Music Ontology [3] provides vocabulary for relating instruments to performances and recordings, but mapping a specific synth to specific recordings still requires external sources such as MusicBrainz⁵, which catalogues gear with uneven coverage. The Studio Ontology [4] could describe the surrounding mixers and effects rather than the instrument itself. Each of these is a reasonable direction but would require substantial additional annotation.

⁵<https://musicbrainz.org/>

7 Connection to Modern Data Semantics Research

The two-pipeline structure of this project surfaces a tension that the academic literature has been actively addressing: manual curation produces high-quality A-Boxes but does not scale, while fully automated population from sources like DBpedia produces broader coverage but with substantial gaps. Two recent threads of research, one on population and one on validation, point at how this trade-off might be narrowed.

The first thread treats ontology population itself as a task that large language models can perform. Babaei Giglou et al. [5] introduce LLMs4OL, an evaluation framework testing nine families of large language models (LLMs) on three core ontology learning tasks: term typing, taxonomy discovery, and non-taxonomic relation extraction. Applied to a domain like hardware electronic instruments, an LLM-based extractor could read Wikipedia article prose directly rather than depending on the brittle DBpedia infobox templates that produced the data-quality issues described in Section 3; this would directly improve *scale* (no human-in-the-loop required for every instrument) and *extensibility* (adapting to a new instrument category would mean a new prompt, not a new schema mapping). However, LLM-generated outputs are not always reliable, so extracted facts would still require validation before they could be added to the ontology.

The second thread targets the validation step. Tsaneva et al. [6] integrate LLMs with human-in-the-loop review in a knowledge graph validation pipeline, finding that combining LLM suggestions with selective human review produces better results than either alone. Applied to the present domain, an LLM could propose which DBpedia infobox values are likely wrong, surface them for review, and otherwise let the runtime SPARQL output stand.

Bringing these two threads together would change how the presented ontology develops. The curated CSV pipeline could shrink to an oversight task on top of an LLM extractor; the runtime SPARQL output's data-completeness gaps could be filled by an LLM reading Wikipedia text directly; the manual verification step could be augmented by an LLM-driven validator like the one Tsaneva et al. describe. The literature suggests that the combination of these techniques, applied within a human-defined logical structure, is where current research is heading.

8 Acknowledgments

The ontology design, including the choice of domain, the T-Box structure, the defined classes, and the SWRL rules, was carried out by the author. AI assistance from a large language model was used during development to help draft parts of the Python scripts used in the project.

9 Code and files

The submission contains the following directories and files:

- `populate_from_CSV/`, containing the curated CSV pipeline, the verified seed file `abox_candidates_verified.csv`, `populate.py` for generating the A-Box from the CSV, `query.py` containing the seven competency queries, and the output ontology `electronic_music_instruments.owl`.
- `populate_from_SPARQL_Alternative/`, containing the runtime SPARQL population pipeline, `populate_from_SPARQL_endpoint.py`, the same `query.py` file, and the output ontology `electronic_music_instruments_from_sparql.owl`.
- `Pre_Curated_CSV_Scripts/`, containing the scraper and the DBpedia coverage-check script used during preparation of the curated CSV.
- `README.md`, containing setup instructions, dependency information, and example commands for running the population and query scripts.

Dependencies are `rdflib`, `SPARQLWrapper`, and `owlrl`.

References

- [1] E. M. von Hornbostel and C. Sachs, “Systematik der musikinstrumente: Ein versuch,” *Zeitschrift für Ethnologie*, vol. 46, pp. 553–590, 1914.
- [2] S. Kolozali, M. Barthet, G. Fazekas, and M. Sandler, “Knowledge representation issues in musical instrument ontology design,” in *Proc. 12th Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, Miami, FL, USA, 2011, pp. 465–470.
- [3] Y. Raimond, S. Abdallah, M. Sandler, and F. Giasson, “The music ontology,” in *Proc. 8th Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, Vienna, Austria, 2007, pp. 417–422.
- [4] G. Fazekas and M. B. Sandler, “The studio ontology framework,” in *Proc. 12th Int. Soc. Music Inf. Retrieval Conf. (ISMIR)*, Miami, FL, USA, 2011.
- [5] H. Babaei Giglou, J. D’Souza, and S. Auer, “LLMs4OL: Large language models for ontology learning,” in *The Semantic Web – ISWC 2023*, ser. Lecture Notes in Computer Science, vol. 14265, Springer, 2023.
- [6] S. Tsaneva, D. Dessì, F. Osborne, and M. Sabou, “Knowledge graph validation by integrating LLMs and human-in-the-loop,” *Information Processing & Management*, vol. 62, no. 5, p. 104 145, 2025.

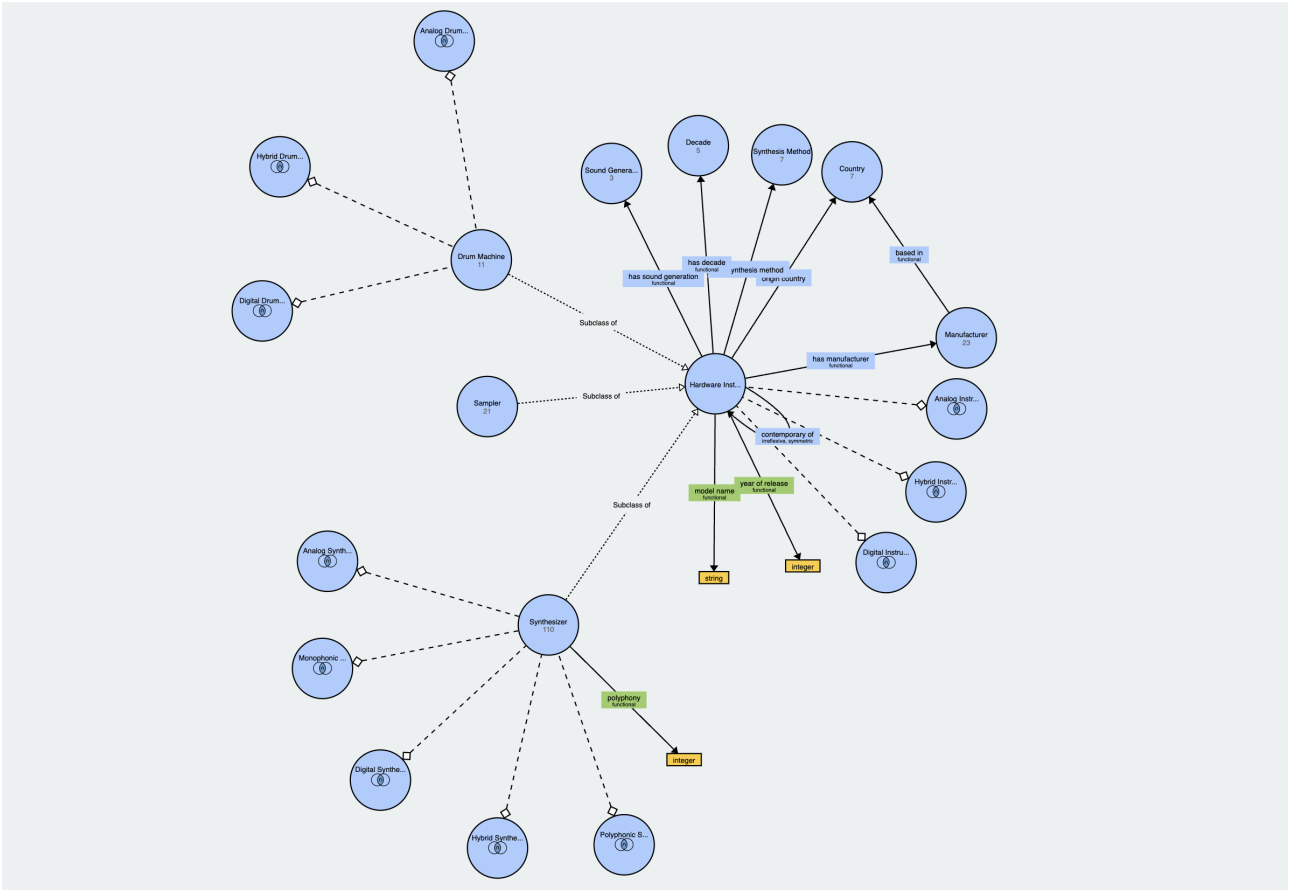


Figure 1: WebVOWL visualisation of the ontology, populated from the curated CSV file.